Summarize

Chapter 15. Build Your Own Trade-Off Analysis

Monday, June 10, 10:01



The conference room somehow seemed more brightly lit than it did on that fateful day in September when the business sponsors of the Sysops Squad were about to pull the plug on the entire support contract business line. People in the conference room were chatting with each other before the meeting started, creating an energy not seen in the conference room for a long, long time.

"Well," said Bailey, the main business sponsor and head of the Sysops Squad ticketing application, "I suppose we should get things started. As you know, the purpose of this meeting is to discuss how the IT department was able to turn things around and repair what was nine months ago a train wreck."

"We call that a retrospective," said Addison. "And it's really useful for discovering how to do things better in the future, and to also discuss things that seemed to work well."

"So then, tell us, what worked really well? How did you turn this business line around from a technical standpoint?" asked Bailey.

"It really wasn't one single thing," said Austen, "but rather a combination of a lot of things. First of all, we in IT learned a valuable lesson about looking at the business drivers as a way to address problems and create solutions. Before, we always used to focus only on the technical aspects of a problem, and as a result never saw the big picture."

"That was one part of it," said Dana, "but one of the things that turned things around for me and the database team was starting to work together more with the application teams to solve problems. You see, before, those of us on the database side of things did our own thing, and the application development teams did their own thing. We never would have gotten to where we are now without collaborating and working together to migrate the Sysops Squad application."

"For me it was learning how to properly analyze trade-offs," said Addison. "If it weren't for Logan's guidance, insights, and knowledge, we wouldn't be in the shape we're in now. It was because of Logan that we were able to justify our solutions from a business perspective."

"About that," said Bailey, "I think I speak for everyone here when I say that your initial business justifications were what prompted us to give you one last shot at repairing the mess we were in. That was something we weren't accustomed to, and, well, quite frankly it took us by surprise—in a good way."

"OK," said Parker, "so now that we all agree things seem to be going well, how do we keep this pace going? How do we encourage other departments and divisions within the company from getting into the same mess we were in before?"

"Discipline," said Logan. "We continue our new habit of creating trade-off tables for all our decisions, continue documenting and communicating our decisions through architecture decision records, and continue collaborating with other teams on problems and solutions."

"But isn't that just adding a lot of extra process and procedures to the mix?" asked Morgan, head of the marketing department.

"No," said Logan. "That's *architecture*. And as you can see, it works."

Throughout this book, the unifying example illustrates how to generically perform trade-off analysis in distributed architectures. However, generic solutions rarely exist in architecture and, if they do, are generally incomplete for highly specific architectures and the unique problems they bring. Thus, we don't think that the communication analysis covered in Chapter 2 is exhaustive, but rather a starting point for you to add more columns for the unique elements entangled with your problem space.

To that end, this chapter provides some advice on how to build your own trade-off analysis, using many of the same techniques we used to derive the conclusions presented in this book.

Our three-step process for modern trade-off analysis, which we introduced in Chapter 2 is as follows:

- Find what parts are entangled together.
- Analyze how they are coupled to one another.
- Assess trade-offs by determining the impact of change to interdependent systems.

We discuss some techniques and considerations for each step next.

Finding Entangled Dimensions

An architect's first step in this process is to discover what dimensions are entangled, or braided, together. This is unique within a particular architecture but discoverable by experienced developers, architects, operations folks, and other roles familiar with the existing overall ecosystem and its capabilities and constraints.

Coupling

The first part of the analysis answers this question for an architect: how are parts within an architecture coupled to one another? The software development world has a wide variety of definitions of coupling, but we use the simplest, most intuitive version for this exercise: if someone changes *X*, will it possibly force *Y* to change?

In Chapter 2, we described the concept of the static coupling between architecture quanta, which provides a comprehensive structural diagram of technical coupling. No generic tool exists to build this because each architecture is unique. However, within an organization, a development team can build a static coupling diagram, either manually or via automation.

For example, to create a static coupling diagram for a microservice within an architecture, an architect needs to gather the following details:

- Operating systems/container dependencies
- Dependencies delivered via transitive dependency management (frameworks, libraries, etc.)
- Persistence dependencies on databases, search engines, cloud environments, etc.
- Architecture integration points required for the service to bootstrap itself
- Messaging infrastructure (such as a message broker) required to enable communication to other quanta

The static coupling diagram does not consider other quanta whose only coupling point is workflow communication with this quantum. For example, if an AssignTicket Service cooperates with the `ManageTicket` within a workflow but has no other coupling points, they are statically independent (but dynamically coupled during the actual workflow).

Teams that already have most of their environments built via automation can build into that generative mechanism an extra capability to document the coupling points as the system builds.

For this book, our goal was to measure the trade-offs in distributed architecture coupling and communication. To determine what became our three dimensions of dynamical quantum coupling, we looked at hundreds of examples of distributed architectures (both microservices and others) to determine the common coupling points. In other words, all the examples we looked at were sensitive to changes to the dimensions of *communication*, *consistency*, and *coordination*.

This process highlights the importance of iterative design in architecture. No architect is so brilliant that their first draft is always perfect. Building sample topologies for workflows (much as we do in this book) allows an architect or team to build a matrix view of trade-offs, allowing quicker and more thorough analysis than ad hoc approaches.

Analyze Coupling Points

Once an architect or team has identified the coupling points they want to analyze, the next step is to model the possible combinations in a lightweight way. Some of the combinations may not be feasible, allowing the architect to skip modeling those combinations. The goal of the analysis is to determine what forces the architect needs to study—in other words, which forces require trade-off analysis? For example, for our architecture quantum dynamic coupling analysis, we chose coupling, complexity, responsiveness/availability, and scale/elasticity as our primary trade-off concerns, in addition to analyzing the three forces of communication, consistency, and coordination, as shown in the ratings table for the "Parallel Saga(aeo) Pattern", appearing again in Table 15-1.

Table 15-1. Ratings for the Parallel Saga pattern

Parallel Saga	Ratings
Communication	Asynchronous
Consistency	Eventual
Coordination	Centralized
Coupling	Low
Complexity	Low
Responsiveness/availability	High
Scale/elasticity	High

When building these ratings lists, we considered each design solution (our named patterns) in isolation, combining them only at the end to see the differences, shown in Table 15-2.

TRADE-OFFS

Table 15-2. Consolidated comparison of dynamic coupling patterns

Pattern	Coupling level	Complexity	Responsiveness/availability	Scale/elasticity
Epic Saga	Very high	Low	Low	Very Low
Phone Tag Saga	High	High	Low	Low
Fairy Tale Saga	High	Very low	Medium	High
Time Travel Saga	Medium	Low	Medium	High
Fantasy Fiction Saga	High	High	Low	Low
Horror Story	Medium	Very high	Low	Medium
Parallel Saga	Low	Low	High	High
Anthology Saga	Very low	High	High	Very high

Once we had analyzed each pattern independently, we created a matrix to compare the characteristics, leading to interesting observations. First, notice the direct inverse correlation between *coupling level* and *scale/elasticity*: the more coupling present in the pattern,

the worse its scalability. This intuitively makes sense; the more services involved in a workflow, the more difficult for an architect to design for scale.

Second, we made a similar observation around *responsiveness/availability* and *coupling level*, which is not quite as direct as the preceding correlation but also significant: higher coupling leads to less responsiveness and availability because the more services involved in a workflow, the more likely the entire workflow will fail based on a service failure.

This analysis technique exemplifies *iterative* architecture. No architect, regardless of their cleverness, can instantly understand the nuances of a truly unique situation—and these nuances constantly present themselves in complex architectures. Building a matrix of possibilities informs the modeling exercises an architect might want to do in order to study the implications of permutating one or more dimensions to see the resulting effect.

Assess Trade-Offs

Once you have built a platform that allows iterative "what if" scenarios, focus on the fundamental trade-offs for a given situation. For example, we focused on synchronous versus asynchronous communication, a choice that creates a host of possibilities and restrictions—everything in software architecture is a trade-off. Thus, choosing a fundamental dimension like synchronicity first limits future choices. With that dimension now fixed, perform the same kind of iterative analysis on subsequent decisions encouraged or forced by the first. An architect team can iterate on this process until they have solved the difficult decisions—in other words, decisions with entangled dimensions. What's left is design.

Trade-Off Techniques

Over time, the authors have created a number of trade-off analyses and have built up some advice on how to approach them.

Qualitative Versus Quantative Analysis

You may have noticed that virtually none of our trade-off tables are *quantitative*—based on numbers—but are rather *qualitative*—measuring the quality of something rather than the quantity, which is necessary because two architectures will always differ enough to prevent true quantitative comparisons. However, using statistical analysis over a large data set allows reasonable qualitative analysis.

For example, when comparing the scalability of patterns, we looked at multiple different *implementations* of communication, consistency, and coordination combinations, assessing

scalability in each case, and allowing us to build the comparative scale shown in Table 15-2.

Similarly, architects within a particular organization can carry out the same exercise, building a dimensional matrix of coupled concerns, and look at representative examples (either within the existing organization or localized spikes to test theories).

We recommend you hone the skill of performing qualitative analysis, as few opportunities for true quantitative analysis exist in architecture.

MECE Lists

It is important for architects to be sure they are comparing the same things rather than wildly different ones. For example, it's not a valid comparison to compare a simple message queue to an enterprise service bus, which contains a message queue but dozens of other components as well.

A useful concept borrowed from the technology strategy world to help architects get the correct match of things to compare is a *MECE list*, an acronym for *mutually exclusive, collectively exhaustive*:

Mutually exclusive

None of the capabilities can overlap between the compared items. As in the preceding example, it is invalid to compare a message queue to an entire ESB because they aren't really the same category of thing. If you want to compare just the messaging capabilities absent the other parts, that reduces the comparison to two mutually comparable things.

Collectively exhaustive

This suggests that you've covered all the possibilities in the decision space, and that you haven't left out any obvious capabilities. For example, if a team of architects is evaluating high-performance message queues and consider only an ESB and simple message queue but not Kafka, they haven't considered all the possibilities in the space.

The goal of a MECE list is to cover a category space completely, with no holes or overlaps, as shown pictorially in Figure 15-1.

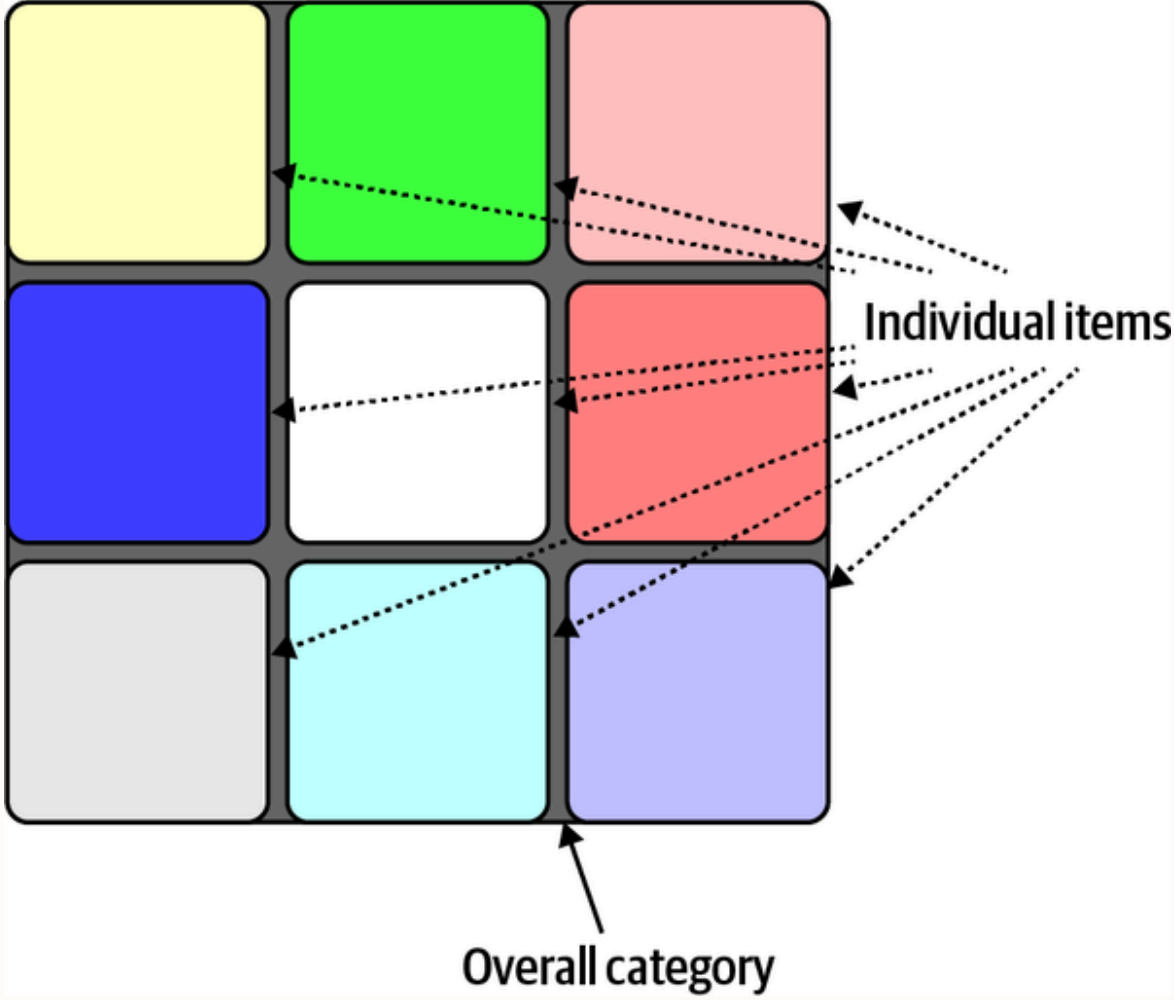


Figure 15-1. A MECE list is mutually exclusive and collectively exhaustive

The software development ecosystem constantly evolves, uncovering new capabilities along the way. When making a decision with long-term implications, an architect should make sure a new capability hasn't just arrived that changes the criteria. Ensuring that comparison criteria is collectively exhaustive encourages that exploration.

The "Out-of-Context" Trap

When assessing trade-offs, architects must make sure to keep the decision in context; otherwise, external factors will unduly affect their analysis. Often, a solution has many beneficial aspects, but lacks critical capabilities that prevent success. Architects need to make sure they balance the *correct* set of trade-offs, not all available ones.

For example, perhaps an architect is trying to decide whether to use a shared service or shared library for common functionality within a distributed architecture, as illustrated in Figure 15-2.

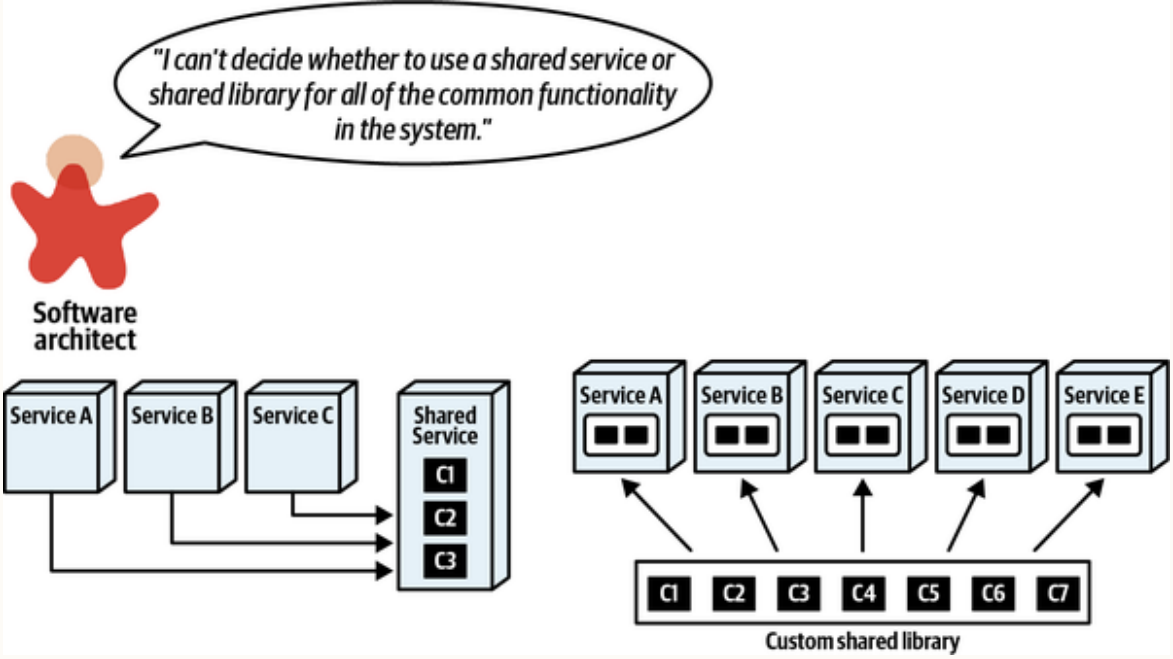
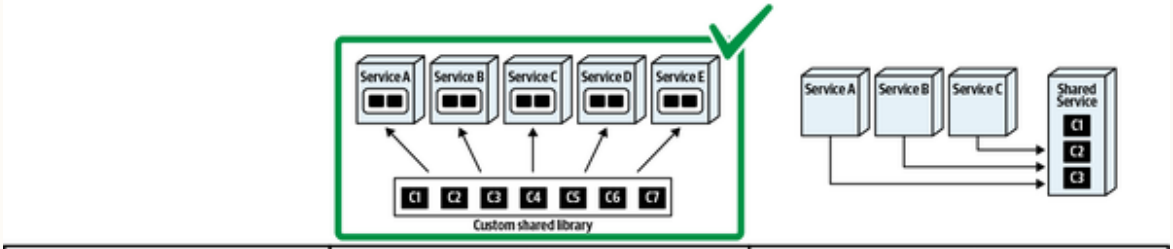


Figure 15-2. Deciding between shared *service* or *library* in a distributed architecture

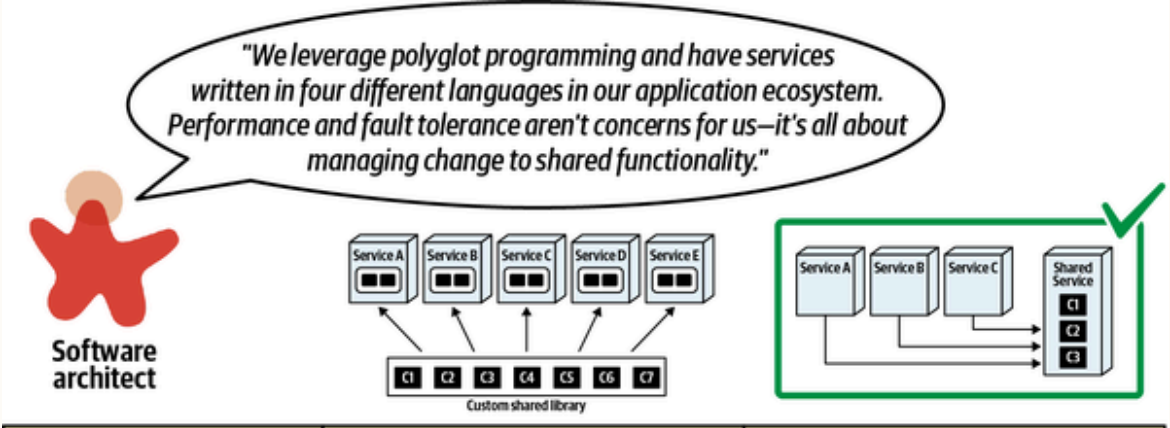
The architect facing this decision will begin to study the two possible solutions, both via general characteristics discovered through research and via experimental data from within their organization. The results of that discovery process lead to a trade-off matrix such as the one shown in Figure 15-3.



heterogeneous code	✗	✓
high code volatility	✗	✓
ability to version changes	✓	✗
dependency management	✗	✓
overall change risk	✓	✗
performance	✓	✗
fault tolerance	✓	✗
scalability	✓	✗

Figure 15-3. Trade-off analysis for two solutions

The architect seems justified in choosing the shared library approach, as the matrix clearly favors that solution...*overall*. However, this decision exemplifies the *out-of-context* problem—when the extra context for the problem becomes clear, the decision criteria changes, as illustrated in Figure 15-4.



heterogeneous code	✗	✓
high code volatility	✗	✓
ability to version changes	✓	✗
dependency management	✗	✓
overall change risk	✓	✗
performance	✓	✗
fault tolerance	✓	✗
scalability	✓	✗

Figure 15-4. Shifting decision based on additional context

The architect continued to research not only the generic problem of service versus library, but the actual context that applies in this situation. Remember, generic solutions are rarely useful in real-world architectures without applying additional situation-specific context.

This process emphasizes two important observations. First, finding the best context for a decision allows the architect to consider fewer options, greatly simplifying the decision process. One common piece of advice from software sages is "embrace simple designs," without ever explaining how to achieve that goal. Finding the correct *narrow* context for decisions allows architects to think about less, in many cases simplifying design.

Second, it's critical for architects to understand the importance of *iterative* design in architecture, diagramming sample architectural solutions to play qualitative "what-if" games to see how architecture dimensions impact one another. Using iterative design, architects can investigate possible solutions and discover the proper context in which a decision belongs.

Model Relevant Domain Cases

Architects shouldn't make decisions in a vacuum, without relevant drivers that add value to the specific solution. Adding those domain drivers back to the decision process can help the architect filter the available options and focus on the really important trade-offs.

For example, consider this decision by an architect as to whether to create a single payment service or a separate service for each payment type, as illustrated in Figure 15-5.

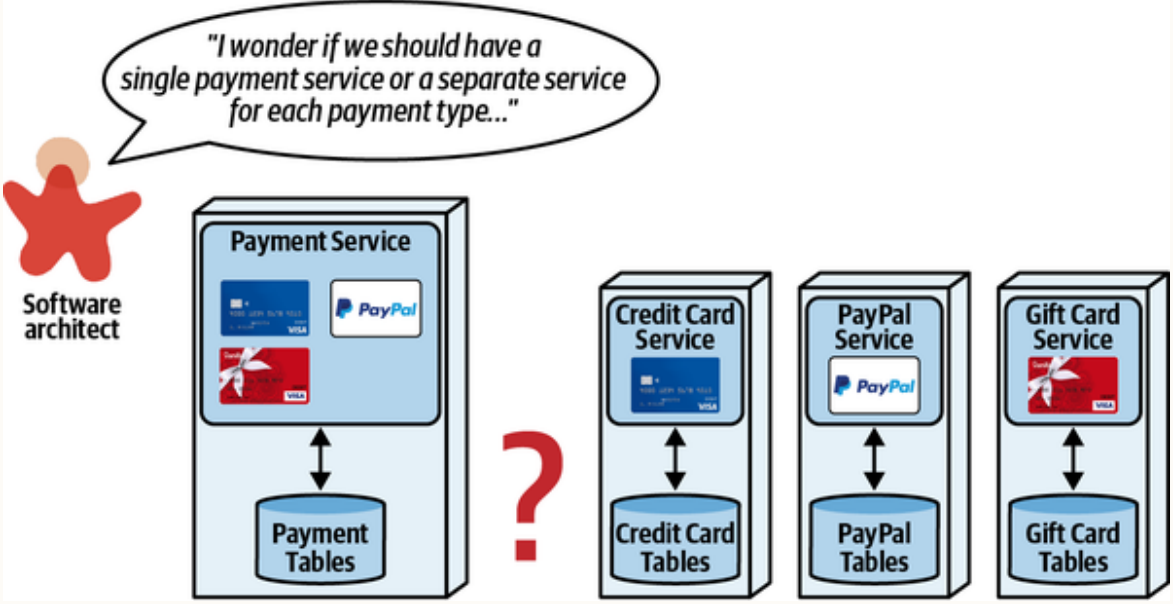


Figure 15-5. Choosing between a single payment service or one per payment type

As we discussed in Chapter 7, architects can choose from a number of integrators and disintegrators to assist this decision. However, those forces are generic—an architect may add more nuance to the decision by modeling some likely scenarios.

For example, consider the first scenario, illustrated in Figure 15-6, to update a credit card processing service.

In this scenario, having separate services provides better *maintainability*, *testability*, and *deployability*, all based on quantum-level isolation of the services. However, the downside of separate services is often duplicated code to prevent static quantum coupling between the services, which damages the benefit of having separate services.

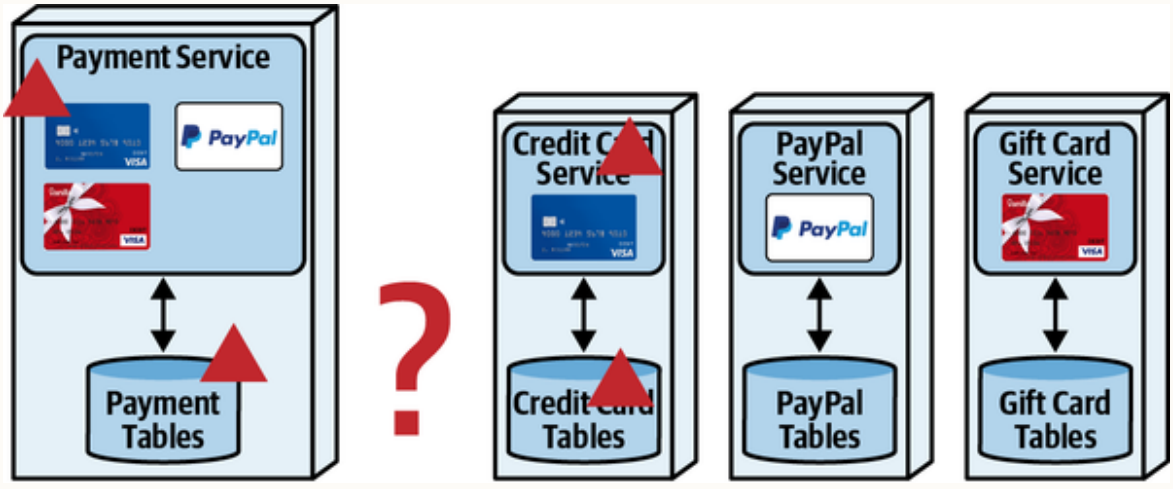


Figure 15-6. Scenario 1: update credit card processing service

In the second scenario, the architect models what happens when the system adds a new payment type, as shown in Figure 15-7.

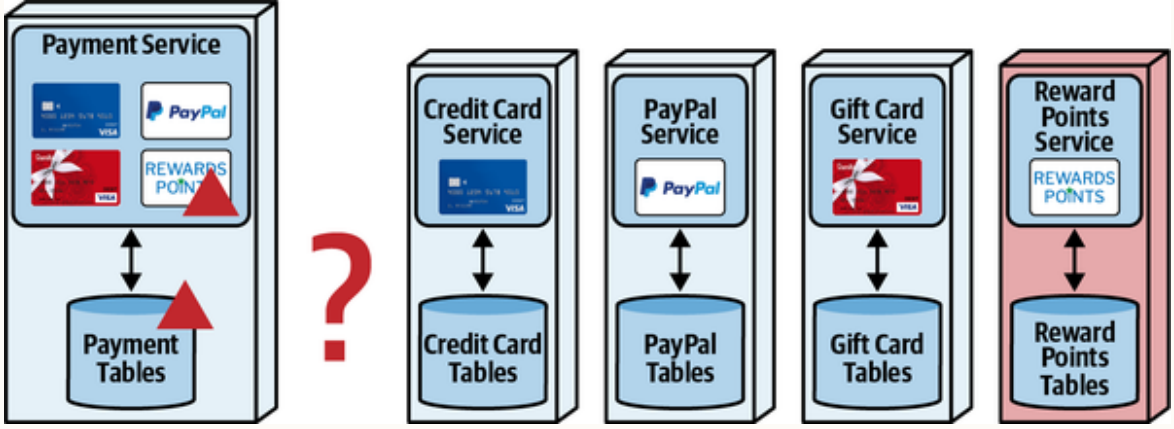


Figure 15-7. Scenario 2: adding a payment type

The architect adds a *reward points* payment type to see what impact it has on the architecture characteristics of interest, highlighting *extensibility* as a benefit of separate services. So far, separate services look appealing.

However, as in many cases, more complex workflows highlight the difficult parts of the architecture, as shown in the third scenario in Figure 15-8.

In this scenario, the architect starts gaining insight into the real trade-offs involved in this decision. Utilizing separate services requires coordination for this workflow, best handled by an orchestrater. However, as we discussed in Chapter 11, moving to an orchestrator likely impacts performance negatively and makes data consistency more of a challenge. The architect could avoid the orchestrator, but the workflow logic must reside somewhere—remember, semantic coupling can only be increased via implementation, never decreased.

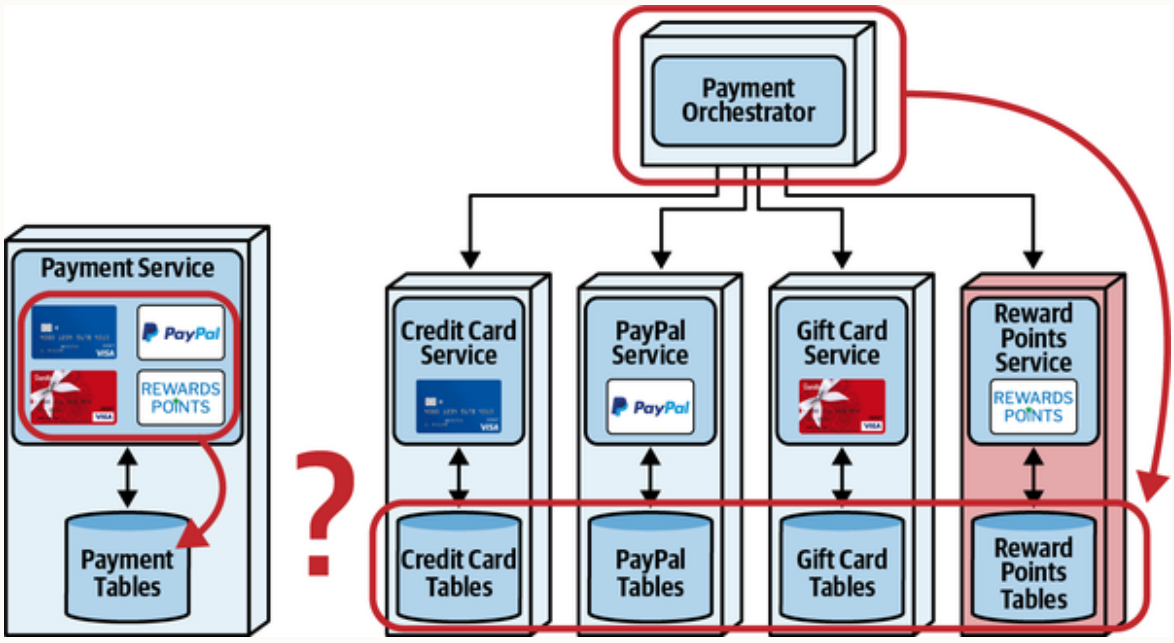


Figure 15-8. Scenario 3: using multiple types for payment

Having modeled these three scenarios, the architect realizes that the real trade-off analysis comes down to which is more important: *performance and data consistency* (a single payment service) or *extensibility and agility* (separate services).

Thinking about architecture problems in the generic and abstract gets an architect only so far. As architecture generally evades generic solutions, it is important for architects to build their skills in modeling relevant domain scenarios to home in on better trade-off analysis and decisions.

Prefer Bottom Line over Overwhelming Evidence

It's easy for architects to build up an enormous amount of information in pursuit of learning all the facets of a particular trade-off analysis. Additionally, anyone who learns something new generally wants to tell others about it, especially if they think the other party will be interested. However, many of the technical details that architects uncover are arcane to nontechnical stakeholders, and the amount of detail may overwhelm their ability to add meaningful insight into the decision.

Rather than show all the information they have gathered, an architect should reduce the trade-off analysis to a few key points, which are sometimes aggregates of individual trade-offs.

Consider the common problem an architect might face in a microservices architecture about the choice of synchronous or asynchronous communication, illustrated in Figure 15-9.

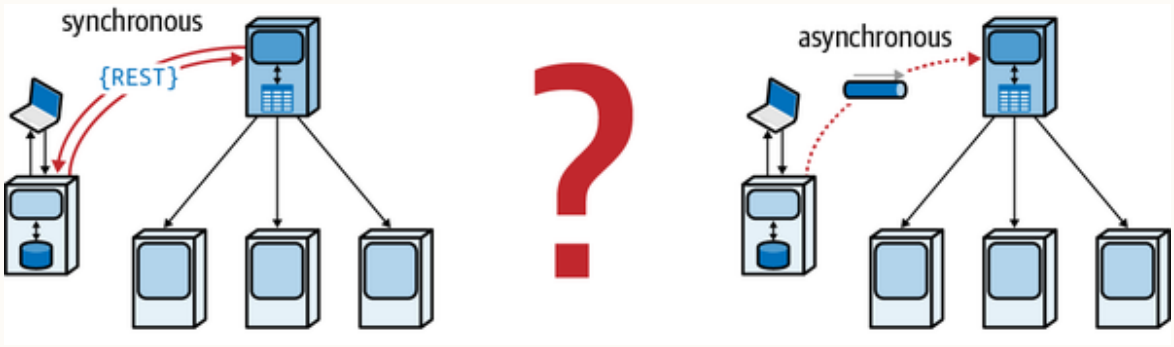


Figure 15-9. Deciding between communication types

The synchronous solution orchestrator makes synchronous REST calls to communicate with workflow collaborators, whereas the asynchronous solution uses message queues to implement asynchronous communication.

After considering the generic factors that point to one versus the other, the architect next thinks about specific domain scenarios of interest to nontechnical stakeholders. To that end, the architect will build a trade-off table that resembles Table 15-3.

TRADE-OFFS

Table 15-3. Trade-offs between synchronous and asynchronous communication for credit card processing

Synchronous advantage	Synchronous disadvantage	Asynchronous advantage	Asynchronous disadvantage
	Customer must wait for credit card approval process to start	No wait for process to start	
Credit approval is guaranteed to start before customer request ends			No guarantee that the process has started
	Customer application rejected if orchestrator is down	Application submission not dependent on orchestrator	

After modeling these scenarios, the architect can create a bottom-line decision for the stakeholders: which is more important, *a guarantee that the credit approval process starts immediately* or *responsiveness and fault-tolerance*? Eliminating confusing technical details allows the nontechnical domain stakeholders to focus on outcomes rather than design decisions, which help avoids drowning them in a sea of details.

Avoiding Snake Oil and Evangelism

One unfortunate side effect of enthusiasm for technology is evangelism, which should be a luxury reserved for tech leads and developers but tends to get architects in trouble.

Trouble comes because, when someone evangelizes a tool, technique, approach, or anything else people build enthusiasm for, they start enhancing the good parts and diminishing the bad parts. Unfortunately, in software architecture, the trade-offs always eventually return to complicate things.

An architect should also be wary of any tool or technique that promises any shocking new capabilities, which come and go on a regular basis. Always force evangelists for the tool or technique to provide an honest assessment of the good and bad—nothing in software architecture is all good—which allows a more balanced decision.

For example, consider an architect who has had success in the past with a particular approach and becomes an evangelist for it, as illustrated in Figure 15-10.

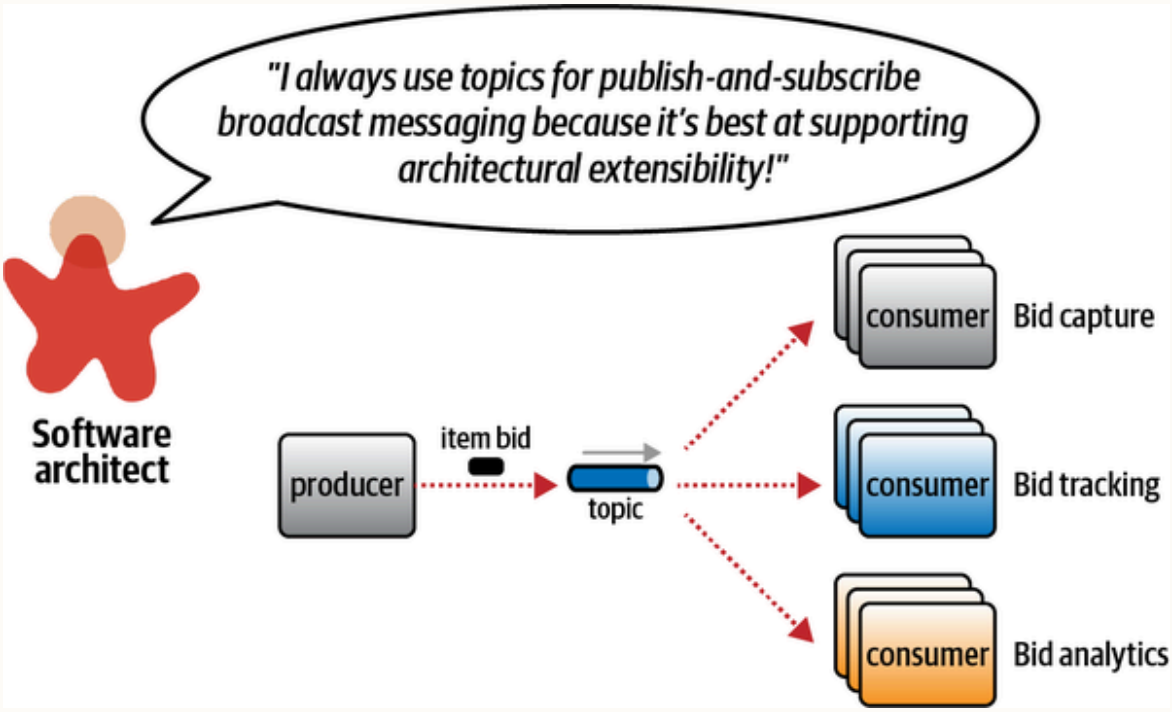


Figure 15-10. An architect evangelist who thinks they have found a silver bullet

This architect has likely worked on problems in the past where extensibility was a key driving architecture characteristic and believes that capability will always drive the decision process. However, solutions in architecture rarely scale outside narrow confines of a particular problem space. On the other hand, anecdotal evidence is often compelling. How do you get to the real trade-off hiding behind the knee-jerk evangelism?

While experience is useful, scenario analysis is one of an architect's most powerful tools to allow iterative design without building whole systems. By modeling likely scenarios, an architect can discover if a particular solution will, in fact, work well.

In the example shown in Figure 15-10, an existing system uses a single topic to broadcast changes. The architect's goal is to add *bid history* to the workflow—should the team keep the existing publish-and-subscribe approach or move to point-to-point messaging for each consumer?

To discover the trade-offs for this specific problem, the architect should model likely domain scenarios using the two topologies. Adding bid history to the existing publish-and-subscribe design appears in Figure 15-11.

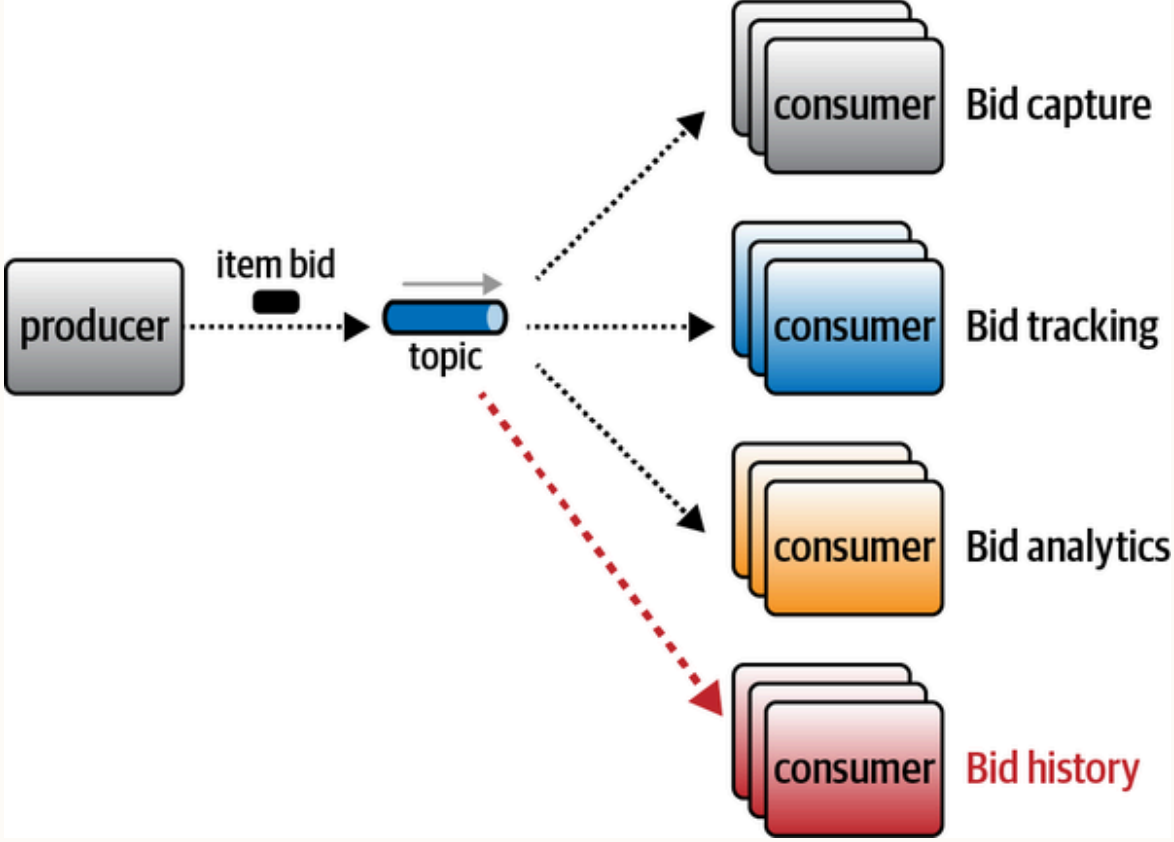


Figure 15-11. Scenario 1: Adding *bid history* to the existing topic

While this solution works, it has issues. First, what if the teams need different contracts for each consumer? Building a single large contract that encompasses everything implements the "Stamp Coupling for Workflow Management" anti-pattern; forcing each team to unify on a single contract creates an accidental coupling point in the architecture—if one team changes its required information, all the teams must coordinate on that change. Second, what about data security? Using a single publish-and-subscribe topic, each consumer has access to all the data, which can create both security problems and PII (Personally Identifiable Information, discussed in Chapter 14) issues as well. Third, the architect should consider the operational architecture characteristic differences between the different consumers. For example, if the operations team wanted to monitor queue depth and use auto-scaling for *bid capture* and *bid tracking* but not for the other two services, using a single topic prevents that capability—the consumers are now operationally coupled together.

To mitigate these shortcomings, the architect should model the alternative solution to see if it addresses the preceding problems (and doesn't introduce new intractable ones). The individual queue version appears in Figure 15-12.

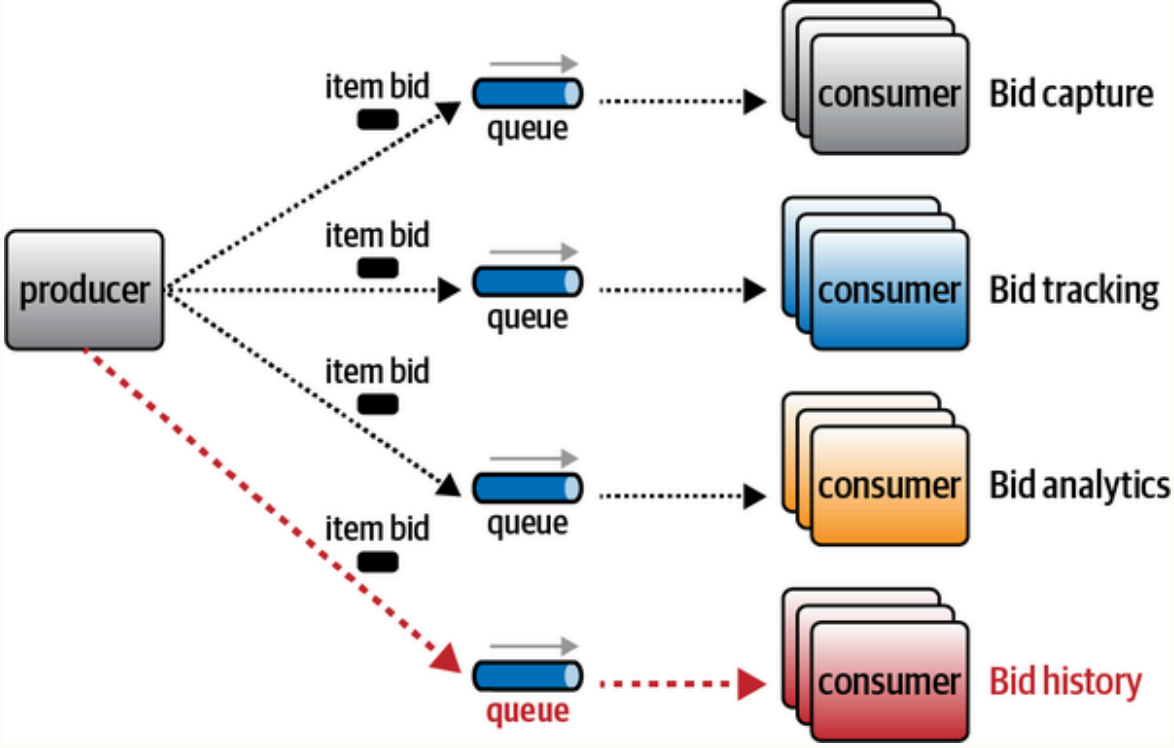


Figure 15-12. Using individual queues to capture bid information

Each part of this workflow (bid capture, bid tracking, bid analytics, and bid history) utilizes its own message queues and addresses many of the preceding problems. First, each consumer can have their own contract, decoupling the consumers from each other. Second, security access and control of data resides within the contract between the producer and each consumer, allowing differences in both information and rate of change. Third, each queue can now be monitored and scaled independently.

Of course, by this point in the book, you should realize that the point-to-point based system isn't perfect either but offers a different set of trade-offs.

Once the architect has modeled both approaches, it seems that the differences boil down to the choices shown in Table 15-4.

TRADE-OFFS

Table 15-4. Trade-offs between point-to-point versus publish-and-subscribe messaging

Point-to-point	Publish-and-subscribe
Allows heterogeneous contracts	Extensibility (easy to add new consumers)
More granular security access and data control	
Individual operational profiles per consumer	

In the end, the architect should consult with interested parties (operations, enterprise architects, business analysts, and so on) to determine which of these sets of trade-offs is more important.

Sometimes an architect doesn't choose to evangelize something but is rather coerced into playing an opposite foil, particularly for something that has no clear advantage. Technologies develop fans, sometimes fervent ones, who tend to downplay disadvantages and enhance upsides.

For example, recently a tech lead on a project tried to wrangle one of the authors into an argument about monorepo versus trunk-based development. Both have good and bad aspects, a classic software architecture decision. The tech lead was a fervent supporter of the monorepo approach, and tried to force the author to take the opposing position—it's not an argument if two sides don't exist.

Instead, the architect pointed out that it was a trade-off, gently explaining that many of the advantages touted by the tech lead required a level of discipline that had never manifested within the team in the past, but will surely improve.

Rather than be forced into taking the opposing position, instead the architect forced a real-world trade-off analysis, not based on generic solutions. The architect agreed to try the Monorepo approach but also gather metrics to make sure that the negative aspects of the solution didn't manifest. For example, one of the damaging anti-patterns they wanted to avoid was accidental coupling between two projects because of repository proximity, so the architect and team built a series of fitness functions to ensure that, while technically possible to create a coupling point, the fitness function prevented it.

TIP

Don't allow others to force you into evangelizing something—bring it back to trade-offs.

We advise architects to avoid evangelizing and to try to become the objective arbiter of trade-offs. An architect adds real value to an organization not by chasing silver bullet after silver bullet but rather by honing their skills at analyzing the trade-offs as they appear.

Sysops Squad Saga: Epilogue

Monday, June 20, 16:55



"OK, I think I finally get it. We can't really rely on generic advice for our architecture—it's too different from all the others. We have to do the hard work of trade-off analysis constantly."

"That's correct. But it's not a disadvantage—it's an advantage. Once we all learn how to isolate dimensions and perform trade-off analysis, we're learning concrete things about *our* architecture. Who cares about other, generic ones? If we can boil the number of trade-offs for a problem down to a small enough number to actually model and test them, we gain invaluable knowledge about our ecosystem. You know, structural engineers have built a ton of math and other predictive tools, but building their stuff is difficult and expensive. Software is a lot...well, softer. I've always said that *testing is the engineering rigor of software development*. While we don't have the kind of math other engineers have, we can incrementally build and test our solutions, allowing much more flexibility and leveraging the advantage of a more flexible medium. Testing with objective outcomes allows our trade-off analyses to go from qualitative to quantitative—from speculation to engineering. The more concrete facts we can learn about our unique ecosystem, the more precise our analysis can become."

"Yeah, that makes sense. Want to go to the after-work gathering to celebrate the big turnaround?"

"Sure."

Previous chapter
< 14. Managing Analytical Data

Next chapter
A. Concept and Term References >